SAFU: System Architecture

safustaking.com · Deterministic loss-socialization protocol

● Live today ○ Architecture in progress, not yet deployed SAME marks a component that is the literal same running instance across both models

● SYSTEM TOPOLOGY

Nodes, computation, and state, staker to blockchain

The real infrastructure path a claim runs through, independent of which coverage model triggered it.

	TOPOLOGY
NODES	Ethereum RPC / archive node and Stellar Horizon / RPC node (staking + claim reads, both chains) · AWS KMS (signing hardware) · VPS (Claim Processor host)
COMPUTATIONAL	Claim Processor receives the request → Detection Engine scores it, EVM and Stellar both → Tier Assessment + Entitlement calc run → KMS Oracle Signer produces a signed attestation for the target chain
STATE	Off-chain: Redis (fast, 90d TTL) + SQLite (durable). On-chain: SAFUPoolV8.sol (Ethereum mainnet) and the Soroban ProtectionPool contract (Stellar testnet) each hold their own stake, beneficiaryHash, and tier records.
TEST VS PRODUCTION	Solidity / EVM layer: Forge test suite 75/75, Halmos symbolic verification 10/10. Soroban / Stellar layer: 184/184 tests passing.

● PRODUCT COMPARISON

One core, two coverage models

Retail wallet coverage is live today. Protocol-level coverage reuses the same claim processor, the same Detection Engine, and the same on-chain contract, read down any row to see what changes and what doesn't.

	RETAIL · LIVE TODAY	PROTOCOL POOL · ARCHITECTURE IN PROGRESS
USER CONTROL	**Staker** Wallet or agent. Designates a beneficiary at stake time, only that beneficiary can later claim.	**BD-attested staking identity** Protocol's designated address, bound to a real identity through onboarding. Never inferred from on-chain data.
APPLICATION	**Claim Processor** SAME VPS-hosted, deterministic claim-processing service	**Claim Processor** SAME Same service, protocol claims route through the same processor
PROTOCOL	**SAFUPoolV8.sol** SAME Ethereum mainnet, live `0xa170f0937DEc353C1806eaC0c3d559524d458641`	**SAFUPoolV8.sol** SAME Same contract, same stakeETH(beneficiary,...) shape as retail
COMPONENT	**Detection Engine** SAME Hacks covered: phishing, approval exploits, key compromise, MEV sandwich attacks. Catastrophic-loss check, detection confidence gate, vesting gate. **2-of-2 override** SAME Optional escape hatch, not a normal-claim step. Standard claims are approved by the oracle alone, this only activates to correct or expedite a specific case. **KMS Oracle Signer** SAME secp256k1, key never leaves KMS hardware **Persistence** Redis (fast, 90d TTL) + SQLite (durable)	**Detection Engine** SAME Proprietary, closed-source, built for EVM and Stellar. Fires on a verified exploit: oracle manipulation, invariant violation, governance manipulation, privileged-function abuse, key compromise, bridge compromise, simple wallet compromise. **Tier Assessment** Objective, computed at claim time from verified TVL, audit history, and depositor base. Fails closed on any data outage. **Entitlement calc** min(protocol stake × tier ratio, real measured loss) **KMS Oracle Signer** SAME **& 2-of-2 override** SAME Same signer, same escape hatch as retail
TRANSFER	**Payout stream** SAME Pull-payment, on-chain tier-cap backstop enforces the cap regardless of off-chain computation, gated by beneficiaryHash	**Payout stream** SAME Same mechanism, gated by beneficiaryHash, only the protocol-designated beneficiary can claim, no outside party can
TECH STACK	Python claim pipeline · Solidity (SAFUPoolV8) · AWS KMS · VPS infrastructure	Same stack, same infrastructure

● **Ethereum** production, mainnet ● **Stellar** testnet only, SCF #44

● REQUEST PATH

Staker to blockchain, at a glance

Staker or Protocol → HTTPS → Claim Processor → Detection Engine → KMS Oracle Signer → SAFUPoolV8.sol / Soroban ProtectionPool

Live today Architecture in progress, not deployed

Pool logic (staking rules, tier thresholds, entitlement formulas) is deployer-flexible, built on SAFU's initial logic or customized per ecosystem. The Detection Engine is never deployer-owned or reimplemented, it stays SAFU-built and closed-source across every deployment.